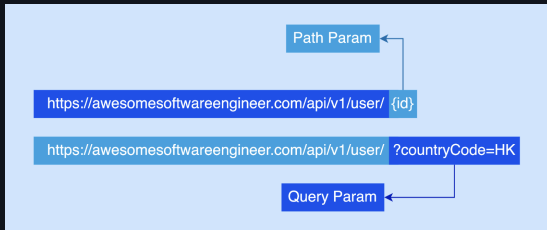


NestJs



Mitsiu Alejandro Carreño Sarabia

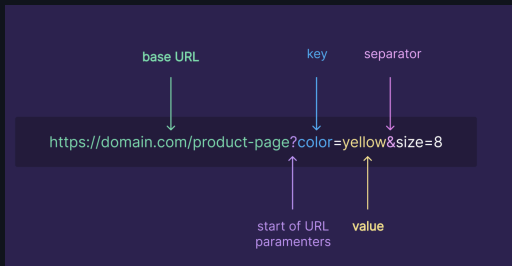
■ Course structure Partial test

- 2nd Partial = 30%
 - Autoevaluation = 5%
 - Theoretical evaluation = 40%
 - Practical evaluation = 55%

Agenda

- Query params
- Route params
- Pipes
- Lab

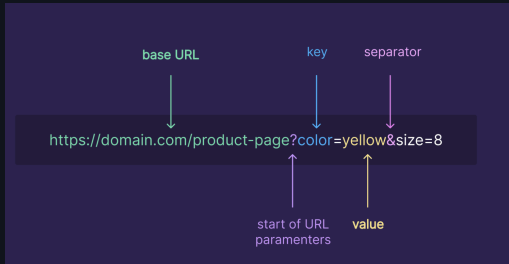
Query Params



Query parameters live after the `?` and tell the server how to tailor the response.

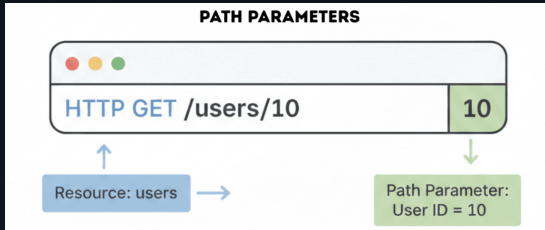
Allows the user to provide extra information about the resource being requested.

Query Params



```
@Controller('product-page')
export class ProductController{
  @Get()
  getProduct(@Query('color') color:string, @
  Query('size') size:string):string[]{
    return ["client requested", color, size];
  }
}
```

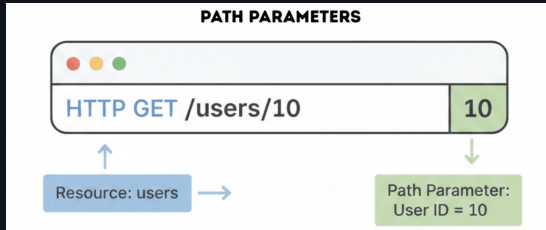
Route Params



Are dynamic URL segments

Also allows the user to provide extra information about the resource being requested.

Route Params

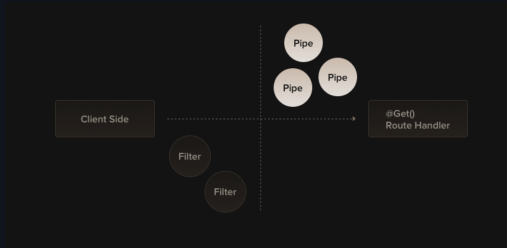


Are defined with a colon (:)

```
@Controller('users')
export class UserController {
  @Get(':id')
  getUser(@Param('id') id:string): string {
    return "Requested id " + id;
  }
}
```

Pipes

A pipe is a class annotated with the `@Injectable()` decorator, which implements the `PipeTransform` interface.

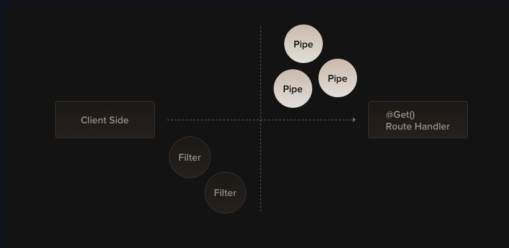


Pipes have two typical use cases:

- transformation: transform input data to the desired form (e.g., from string to integer)
- validation: evaluate input data and if valid, simply pass it through unchanged; otherwise, throw an exception



Pipes

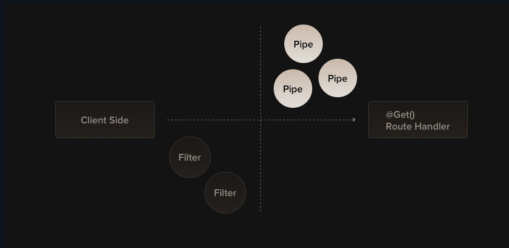


Nest comes with several pipes available out-of-the-box:

- ValidationPipe
- ParseIntPipe
- ParseFloatPipe
- ParseBoolPipe
- ParseArrayPipe
- ParseUUIDPipe
- ParseEnumPipe
- DefaultValuePipe
- ParseFilePipe
- ParseDatePipe



Pipes



```
@Controller('users')
export class UserController{
  @Get(':id')
  getUser(@Param('id', ParseIntPipe) id:number
): number {
    return id + 2;
  }
}
```